

Limit-Pushing Hacker Playbook: Bypassing Engine Constraints:

To achieve mechanics normally thought impossible on Bedrock, you can twist engine subsystems to serve unintended purposes.

1. Simulated Tile Entities & Arbitrary Block NBT

- Engine Limitation: Custom blocks cannot hold arbitrary persistent data or tick dynamic scripts directly without performance drops.
- The Bypass: The Marker Entity Anchor.

When a custom block is placed, use `world.afterEvents.playerPlaceBlock` to spawn an invisible, intangible marker entity inside the block center (`minecraft:is_hidden_when_invisible`, zero collision, zero gravity). Store all data (energy, contents, fluids, machine state) using `entity.setDynamicProperty()`. When the block breaks, read and delete the marker entity.

<content "[typescript112.ts](#)">

2. Custom GUI & Container Screens Without Native Support

- Engine Limitation: Bedrock JSON doesn't let you design custom inventory window slots for custom blocks.
- The Bypass: Cancelling Interactions into Server-UI Forms.

Listen to `world.beforeEvents.playerInteractWithBlock`. Check if the block is your custom machine. Cancel the event (`event.cancel = true`) to prevent default block interaction, then immediately pipe the player to a scripted `ActionFormData` or `ModalFormData`.

<content "[typescript229.ts](#)">

> UI Styling Tip: Map custom interface borders and inventory boxes to custom Unicode characters inside `font/default_marker.json`. You can print these characters as the title or body text in `@minecraft/server-ui` forms to draw full visual HUDs and interfaces.

3. Zero-Latency Client/Server Synchronization

- Engine Limitation: Scripts execute on the server thread. Running particle math or visual bone transformations solely inside the script loop causes jitter over high-ping connections.
- The Bypass: Actor Property Bridges.

1. Define an Actor Property on the entity in BP JSON:

```
...  
"properties": {  
  "engine:mode": { "type": "int", "range": [0, 5], "default": 0 }  
}  
...
```

2. `entity.setProperty("engine:mode", 2);`

```
...  
entity.setProperty("engine:mode", 2);  
...
```

3. In the Resource Pack's render controller or animation controller, read that property directly in Molang:

```
...  
"transitions": [  
  { "overdrive_state": "q.property('engine:mode') == 2" }  
]  
...
```

The server sets the state once; the client's local graphics engine calculates animations, bone transforms, and sub-tick particle emission at 60+ FPS locally.

4. Custom Raycast Hitboxes & Physics Triggers

- Engine Limitation: Minecraft entities only support standard axis-aligned bounding boxes (AABBs). You cannot native-rotate an entity's hitbox diagonally.
- The Bypass: Vector Math Raycasting.
- For custom weapons, lasers, or directional barriers, avoid relying on vanilla collision. Calculate collisions mathematically per tick using `dimension.getEntitiesFromRay()` and `dimension.getBlockFromRay()`.

<Content "[typescript557.ts](#)">